PM Facilities Platform — GPT Project Roadmap and Alignment Document

Purpose of This Document

This document is the source-of-truth roadmap for the GPT Project that will help build the PM Facilities Work Order Platform.

Every chat inside the GPT Project should use this document to understand:

- What the product is.
- Which phase the project is currently on.
- What should and should not be built in each phase.
- How to avoid scope drift.
- How to structure implementation work.
- How to close out each phase.
- How to prepare the next phase handoff.
- How to keep docs updated so a future chatbot understands the application.

This is not just a feature list. It is the alignment document that future GPT chats should follow.

---

1. Product Summary

The product is a multi-tenant facilities maintenance and project management platform for facilities management companies that act as aggregators between commercial clients and vendors/subcontractors.

The platform will support the three-party facilities workflow:

```
Client / Retailer / Commercial Customer
        ↓
Aggregator / Facilities Maintenance Company
        ↓
Vendor / Subcontractor / Technician
```

The aggregator receives work orders from clients, finds vendors, dispatches vendors, tracks statuses and notes, manages quotes/proposals, receives vendor invoices, marks up or bills client invoices, and reports on performance.

The system must support small service jobs, such as toilet clogs, lock repairs, leaks, HVAC issues, and electrical work, as well as larger project/construction jobs that may require proposals, change orders, progress invoices, and multiple vendors.

2. Core Product Rules

Future GPT chats must preserve these rules.

2.1 Do Not Make the App ServiceChannel-Specific

ServiceChannel is only one external client portal integration. It is not the center of the product.

The system must be source-agnostic and support many work order sources:

```
manual entry
owned client portal
external client portal
ServiceChannel
other client portals
email ingestion
forwarded email
API
preventative maintenance schedule
snow event
```

The app should own the core operating workflow. External portals are input/output channels.

Correct model:

```
Any Work Order Source → Internal Aggregator Platform → Vendor / Client /
Accounting / Analytics
```

Incorrect model:

```
ServiceChannel → App → Vendor
```

2.2 Aggregator Portal Comes First

The internal aggregator portal is the first priority.

Initial users are operators, admins, dispatchers, and accounting users inside the facilities management company.

Vendor portal and client portal are first-class future portals, but they come after the internal aggregator workflow is stable.

2.3 Vendor Portal Is Core to the Long-Term Product

Vendors will eventually log in to update assigned jobs.

Vendor users should be able to:

```
accept dispatch
decline dispatch
confirm schedule
update ETA
mark arrival/on-site
add notes
upload photos
request NTE increase
submit quote
mark work complete
submit invoice
```

Vendor notes and statuses should flow into the aggregator platform first. They should not automatically become client-visible unless business rules allow it.

2.4 Client Portal Is Also First-Class

The platform will also have its own client portal.

Clients should eventually be able to:

```
submit work orders
view work order status
add notes
approve proposals
view invoices
download closeout documents
```

The owned client portal should use the same internal job model as external portal, email, manual, and API-created jobs.

2.5 AI Scope Generation Comes Earlier Than Full AI Automation

The system should support AI-assisted scope-of-work generation sooner than full chatbot automation.

Example client request:

```
Toilet clog
```

Example generated technician scope:

```
Assess affected toilet and surrounding area.
Verify source and severity of clog.
Attempt to clear blockage using standard plumbing methods.
Remove toilet only if required to access blockage.
Clear obstruction and restore normal flushing.
Clean affected work area.
Test toilet operation multiple times before leaving site.
Report additional damage, leaks, or required follow-up repairs.
Upload before and after photos.
```

AI output must be treated as a draft until reviewed or approved by a user.

2.6 Email Ingestion Is a Future Requirement

Some clients submit work orders by email. The system must eventually support email parsing and email-to-work-order intake.

Expected future flow:

```
Client sends or forwards email
        ↓
System stores inbound email
        ↓
Parser extracts client, location, issue, trade, priority, and attachments
        ↓
System creates draft work order intake
        ↓
Operator reviews and approves
        ↓
Draft becomes active job
```

Email parsing should not initially create active jobs without review.

2.7 Analytics Must Be Designed From Day One

Anything important operationally should be tracked historically.

The system must preserve:

```
status history
time in status
priority history
trade changes
notes
vendor updates
client updates
dispatch attempts
invoice events
communication events
integration sync events
user audit logs
```

Do not rely only on current state fields for analytics.

2.8 Every Phase Must Update Documentation

Every phase must update docs so the future chatbot can understand the app.

Each phase should include:

```
phase summary
technical decisions
user SOP
admin SOP
system workflows
business rules
chatbot knowledge
DB changes
API routes
known limitations
closeout
```

---

3. Technical Context

Project folder:

```
~/Desktop/PM
```

Likely terminal prompt:

```
jonnyrosero@Jonnys-MacBook-Pro pm %
```

Database:

```
MySQL / MariaDB hosted on Namecheap
Database name: jonnyrosero_pm
User: jonnyrosero_jonny
Local tunnel host: 127.0.0.1
Local tunnel port: 3307
```

SSH tunnel command:

```
ssh -p 21098 -L 3307:127.0.0.1:3306 jonnyrosero@host62.registrar-servers.com
```

Session-safe MySQL pattern:

```
cd ~/Desktop/PM 2>/dev/null || cd ~/Desktop/pm

read -s MYSQL_PWD
export MYSQL_PWD

mysql --protocol=tcp
  -h 127.0.0.1
  -P 3307
  -u jonnyrosero_jonny
  jonnyrosero_pm
  -e "SELECT DATABASE() AS db_name, NOW() AS server_time;"
```

Do not place the MySQL password directly in shell history.

Recommended stack:

```
Next.js / React
MySQL
Server-side database access only
Git / GitHub
Markdown docs in repo
AI provider abstraction later
External integration adapter pattern later
```

Browser UI should never connect directly to MySQL.

---

4. Recommended Repo Structure

Future chats should keep the project organized like this unless the live repo proves a different structure already exists.

```
pm/
  docs/
    roadmap/
      01-gpt-project-roadmap.md
    phase-0-foundation/
      01-phase-summary.md
      02-decisions.md
      03-user-sop.md
      04-admin-sop.md
      05-system-workflows.md
      06-business-rules.md
      07-chatbot-knowledge.md
      08-db-changes.md
      09-api-routes.md
      10-known-limitations.md
      11-closeout.md
    phase-1-auth-tenancy/
    phase-2-clients-locations/
    phase-3-vendors/
    phase-4-jobs/
    phase-5-dispatch/
    phase-6-communications/
    phase-7-ai-scope-generation/
    phase-8-billing-proposals/
    phase-9-aggregator-dashboard-analytics/
    phase-10-vendor-portal/
    phase-11-client-portal/
    phase-12-external-portal-integrations/
    phase-13-email-ingestion/
    phase-14-preventative-maintenance/
    phase-15-snow-operations/
    phase-16-chatbot-ai-assistant/
  db/
    migrations/
    seeds/
  src/
    app/
```

```
    components/
    lib/
    server/
    types/
```

---

5. GPT Project Working Rules

Every future GPT chat should follow these rules.

5.1 Start by Identifying the Phase

At the beginning of a new implementation chat, determine the active phase from the user's instruction.

If the user says:

```
Start Phase 2
Continue Phase 4
Prepare Phase 7 handoff
Inspect Phase 5
```

then stay within that phase unless the user explicitly changes scope.

5.2 Use This Source-of-Truth Order

When making implementation decisions, use this order:

1. Current user message and current phase instruction.
2. This roadmap document.
3. Current live repo files.
4. Current live database schema.
5. Current phase docs.
6. Prior phase docs for historical context only.

Do not assume the roadmap is more accurate than the live repo or live database once implementation has started. Inspect before changing.

5.3 Work in Small Batches

Future chats should implement in small, verifiable batches.

Preferred batch pattern:

```
inspect current state
propose immediate batch
apply small change
run verification
summarize result
continue only when aligned
```

Do not rewrite large parts of the app without inspecting current files.

5.4 Stay Inside the Current Phase

Do not build future-phase features early unless one of these is true:

```
it is required for the current phase to work
it prevents major rework
it is a harmless schema placeholder
user explicitly requests it
```

Example:

During Phase 4 jobs, it is okay to include `source_type` because future email/client portal/external portal jobs need it.

But do not build the full email parser during Phase 4.

5.5 Preserve Auditability

When implementing workflows, favor historical/event tables over overwriting data only.

Examples:

```
jobs.current_status_id + job_status_history
jobs.priority_id + job_priority_history
job_vendor_assignments.assignment_status + assignment status history
communication_logs for outbound/inbound messages
integration_sync_events for external sync activity
```

5.6 Update Docs Before Phase Closeout

A phase is not complete until docs are updated.

Each phase must include:

```
01-phase-summary.md
02-decisions.md
03-user-sop.md
04-admin-sop.md
05-system-workflows.md
06-business-rules.md
07-chatbot-knowledge.md
08-db-changes.md
09-api-routes.md
10-known-limitations.md
11-closeout.md
```

---

6. Versioning and Git Rules

Use phase-based branches and tags.

Branch examples:

```
phase-0-foundation
phase-1-auth-tenancy
phase-2-clients-locations
phase-3-vendors
phase-4-jobs
```

Tag examples:

```
v0.1.0-phase-0
v0.2.0-phase-1
v0.3.0-phase-2
v0.4.0-phase-3
```

General closeout commands:

```
git status
git add .
git commit -m "Phase X: <short description>"
git tag -a v0.X.0-phase-X -m "v0.X.0 Phase X <phase name>"
git push -u origin phase-X-name
git push origin v0.X.0-phase-X
```

Local snapshot rule before major phases:

```
cd ~/Desktop

rsync -a
  --exclude 'node_modules'
  --exclude '.next'
  --exclude '.git'
  PM/ PM_snapshot_v0_X_0_phase_X/
```

If project folder is lowercase:

```
cd ~/Desktop

rsync -a
  --exclude 'node_modules'
  --exclude '.next'
  --exclude '.git'
  pm/ pm_snapshot_v0_X_0_phase_X/
```

GitHub should be the source of truth. Local snapshots are safety backups.

---

7. Phase Roadmap Overview

Version	Phase	Main Goal
v0.1.0	Phase 0	Foundation, repo, docs, roadmap
v0.2.0	Phase 1	Multi-tenant auth, users, roles
v0.3.0	Phase 2	Clients and client locations
v0.4.0	Phase 3	Vendors, vendor locations, service coverage
v0.5.0	Phase 4	Jobs/work orders foundation
v0.6.0	Phase 5	Dispatch workflow
v0.7.0	Phase 6	Notes, communication, update engine
v0.8.0	Phase 7	AI-assisted scope generation
v0.9.0	Phase 8	Billing, proposals, change orders
v1.0.0	Phase 9	Aggregator dashboard and analytics MVP
v1.1.0	Phase 10	Vendor portal MVP

Version	Phase	Main Goal
v1.2.0	Phase 11	Client portal MVP
v1.3.0	Phase 12	External portal integration framework
v1.4.0	Phase 13	Email-to-work-order ingestion
v1.5.0	Phase 14	Preventative maintenance module
v1.6.0	Phase 15	Snow operations module
v2.0.0	Phase 16	Chatbot and AI operations assistant

---

8. Detailed Phase Plan

Phase 0 — Foundation, Repo, Docs, and Roadmap

Version:

```
v0.1.0-phase-0
```

Goal:

Create the project foundation and documentation structure.

Deliverables:

```
Git repo initialized
project folder confirmed
roadmap saved under docs/roadmap
phase docs structure created
db/migrations created
db/seeds created
source-of-truth rules documented
versioning rules documented
closeout template created
```

Acceptance criteria:

```
Repo has clean folder structure.
Roadmap exists in the repo.
Phase 0 closeout docs exist.
```

```
Git branch exists.
Git commit exists.
Version tag exists.
```

Do not build:

```
auth
clients
vendors
jobs
portals
integrations
AI
```

Phase 1 — Multi-Tenant Auth, Users, and Roles

Version:

```
v0.2.0-phase-1
```

Goal:

Create the multi-tenant foundation so every future record can be tenant-scoped.

Core tables:

```
tenants
users
roles
tenant_users
user_roles
audit_logs
```

Initial roles:

```
super_admin
tenant_admin
operator
accounting
```

```
vendor_user
client_user
```

Deliverables:

```
login/logout flow
protected app shell
tenant-aware user session
initial role model
server-side tenant guard pattern
basic audit log table if practical
phase docs
```

Acceptance criteria:

```
User can log in.
User belongs to tenant.
Protected pages require auth.
Server-side code can identify current tenant.
Future vendor/client roles are represented.
Phase docs updated.
```

Do not build:

```
client management
vendor management
job creation
vendor portal
client portal
```

Phase 2 — Clients and Client Locations

Version:

```
v0.3.0-phase-2
```

Goal:

Allow aggregator users to manage clients and locations.

Core tables:

```
clients
client_contacts
client_locations
client_location_contacts
client_location_hours
client_location_access_notes
client_billing_rules
```

Core screens:

```
/clients
/clients/new
/clients/[id]
/clients/[id]/locations
/clients/[id]/locations/new
```

Deliverables:

```
create client
view client list
view client detail
create client location
view locations by client
tenant-scoped client/location queries
phase docs
```

Acceptance criteria:

```
Operator can create a client.
Operator can create a client location.
Locations are linked to clients.
Locations are tenant-scoped.
Locations can later be selected when creating jobs.
Phase docs updated.
```

Do not build:

```
jobs
vendor dispatch
```

```
client portal login
external portal integration
```

Phase 3 — Vendors, Vendor Locations, and Coverage

Version:

```
v0.4.0-phase-3
```

Goal:

Create a vendor database that supports local vendors and multi-location/national vendors.

Core tables:

```
vendors
vendor_contacts
vendor_locations
vendor_trade_coverage
vendor_service_areas
vendor_rates
vendor_documents
vendor_compliance
vendor_performance_scores
```

Core screens:

```
/vendors
/vendors/new
/vendors/[id]
/vendors/[id]/locations
/vendors/[id]/coverage
```

Deliverables:

```
create vendor
view vendor list
view vendor detail
add vendor contact
add vendor location
```

```
assign vendor trade coverage
assign vendor service area
phase docs
```

Acceptance criteria:

```
Operator can create vendor.
Operator can add one or more vendor locations.
Vendor can support multiple trades.
Vendor coverage can support geographic dispatch later.
Vendor model is ready for future vendor portal users.
Phase docs updated.
```

Do not build:

```
vendor portal login
job dispatch
vendor invoices
```

Phase 4 — Jobs / Work Orders Foundation

Version:

```
v0.5.0-phase-4
```

Goal:

Create the central job/work order object.

Core tables:

```
jobs
job_contacts
job_status_history
job_priority_history
job_trade_history
job_notes
job_attachments
job_events
```

Important job fields:

```
tenant_id
job_number
client_id
client_location_id
primary_trade_id
priority_id
current_status_id
source_type
source_external_id
problem_description
scope_of_work
generated_scope_of_work
approved_scope_of_work
scope_generation_status
not_to_exceed_amount
scheduled_start_at
scheduled_end_at
due_at
completed_at
closed_at
created_by_user_id
created_at
updated_at
```

Source types to allow early:

```
manual
internal_client_portal
external_client_portal
email_ingestion
forwarded_email
api
preventative_maintenance
snow_event
```

Core screens:

```
/jobs
/jobs/new
/jobs/[id]
```

Deliverables:

```
create job from client location
select trade
select priority
enter problem description
enter initial scope or placeholder scope
assign initial status
write status history
write job event
view job detail
phase docs
```

Acceptance criteria:

```
Operator can create a job.
Job links to client and client location.
Job has source_type.
Job has current status.
Status history is recorded.
Job event is recorded.
Job detail page shows core info.
Phase docs updated.
```

Do not build:

```
full dispatch workflow
AI scope generator UI
vendor portal
client portal
email parser
```

Phase 5 — Dispatch Workflow

Version:

```
v0.6.0-phase-5
```

Goal:

Allow operators to assign vendors to jobs.

Core tables:

```
job_vendor_assignments
job_vendor_assignment_status_history
dispatch_messages
vendor_eta_confirmations
vendor_check_ins
vendor_check_outs
```

Dispatch statuses:

```
draft
sent
accepted
declined
scheduled
confirmed
on_site
work_complete
cancelled
```

Deliverables:

```
assign vendor to job
select vendor location/contact
set agreed NTE/DNE
set scheduled date/time
store dispatch scope
update dispatch status
write assignment status history
show dispatch section on job detail
phase docs
```

Acceptance criteria:

```
Operator can dispatch a vendor.
A job can support multiple vendor assignments.
Dispatch status history is tracked.
Dispatch info is visible on job detail.
Approved scope can be used as dispatch scope.
Phase docs updated.
```

Do not build:

```
vendor portal self-service
full email/SMS sending unless needed as simple placeholder
client portal updates
```

Phase 6 — Notes, Communication, and Update Engine

Version:

```
v0.7.0-phase-6
```

Goal:

Make job timeline, notes, and communication history central to the app.

Core tables:

```
job_notes
communication_logs
email_templates
outbound_messages
inbound_messages
client_update_logs
vendor_update_logs
portal_update_queue
```

Note visibility options:

```
internal_only
vendor_visible
client_visible
client_and_vendor_visible
requires_review
```

Communication channels:

```
internal_note
vendor_portal
client_portal
email
sms
```

```
external_portal
phone_call
```

Deliverables:

```
internal notes
client-visible notes
vendor-visible notes
job timeline
communication log structure
basic update queue concept
visibility/review rules
phase docs
```

Acceptance criteria:

```
Operator can add notes to job.
Notes have visibility.
Job timeline shows notes/events/status changes.
Client/vendor visibility is controlled.
Communication records can be tied to jobs.
Phase docs updated.
```

Do not build:

```
full AI update writer
full external portal sync
full vendor portal
full client portal
```

Phase 7 — AI-Assisted Scope Generation

Version:

```
v0.8.0-phase-7
```

Goal:

Help operators generate structured technician scopes from short issue descriptions.

Core tables:

```
scope_templates
scope_template_steps
job_scope_steps
ai_scope_generation_logs
ai_prompt_templates
```

Deliverables:

```
scope template model
scope steps model
AI scope generation endpoint or service
operator review/edit/approve flow
approved scope saved to job
AI generation logging
phase docs
```

Acceptance criteria:

```
Operator can generate draft scope from problem description.
Operator can edit generated scope.
Operator can approve scope.
Approved scope is stored on job.
Generation is logged.
AI output is not treated as final until reviewed.
Phase docs updated.
```

Do not build:

```
full chatbot
full autonomous dispatch
full autonomous client updates
```

Phase 8 — Billing, Proposals, and Change Orders

Version:

```
v0.9.0-phase-8
```

Goal:

Support vendor invoices, client invoices, proposals, and change orders.

Core tables:

```
proposals
proposal_line_items
proposal_approvals
change_orders
change_order_line_items
vendor_invoices
vendor_invoice_line_items
client_invoices
client_invoice_line_items
payment_records
job_billing_events
```

Deliverables:

```
vendor invoice record
client invoice record
multiple invoices per job
basic proposal record
basic change order record
billing events
job billing section
phase docs
```

Acceptance criteria:

```
Vendor invoices are separate from client invoices.
A job can have multiple vendor invoices.
A job can have multiple client invoices.
Proposals/change orders can link to jobs.
Billing events are tracked.
Phase docs updated.
```

Do not build:

```
full accounting system
payment processor integration
advanced margin analytics unless simple
```

Phase 9 — Aggregator Dashboard and Analytics MVP

Version:

```
v1.0.0-phase-9
```

Goal:

Create the first complete internal aggregator MVP.

Core analytics:

```
open jobs by status
open jobs by priority
open jobs by client
open jobs by trade
time in status
time to dispatch
time to scheduled
time to arrival
time to completion
vendor assignment count
invoice pending count
```

Deliverables:

```
/dashboard
aggregator job queue
status cards
priority cards
basic operational analytics
job aging indicators
stalled job indicators
phase docs
```

Acceptance criteria:

```
Aggregator can operate the basic workflow.
Dashboard shows useful live counts.
Job detail contains timeline, dispatch, notes, and billing basics.
Analytics use historical records where possible.
Phase docs updated.
```

Do not build:

```
client portal
vendor portal
external integrations
advanced AI chatbot
```

Phase 10 — Vendor Portal MVP

Version:

```
v1.1.0-phase-10
```

Goal:

Allow vendor users to access and update assigned jobs.

Vendor screens:

```
/vendor/jobs
/vendor/jobs/[id]
/vendor/invoices/new
/vendor/profile
```

Deliverables:

```
vendor user login/access
vendor assigned jobs list
vendor job detail
accept/decline dispatch
confirm schedule
add vendor note
update ETA/status
upload photo placeholder if practical
```

```
submit invoice placeholder or basic form
operator review of vendor updates
phase docs
```

Acceptance criteria:

```
Vendor can see assigned jobs only.
Vendor can update assigned job status/details.
Vendor notes are captured as vendor-originated.
Vendor updates do not automatically become client-facing unless allowed.
Operator can review vendor updates.
Phase docs updated.
```

Do not build:

```
client portal
external portal sync
full AI automation
```

Phase 11 — Client Portal MVP

Version:

```
v1.2.0-phase-11
```

Goal:

Allow client users to submit and view work orders through the owned client portal.

Client screens:

```
/client/jobs
/client/jobs/new
/client/jobs/[id]
/client/locations
/client/invoices
```

Deliverables:

```
client user access
client work order submission
client job list
client job detail
client-visible updates
proposal approval placeholder or basic flow
invoice visibility placeholder or basic flow
phase docs
```

Acceptance criteria:

```
Client can submit work order.
Submitted work order enters internal aggregator workflow.
Client sees only client-visible data.
Client-visible updates are controlled by rules.
Operator can manage client-submitted jobs.
Phase docs updated.
```

Do not build:

```
external portal integrations
email parser
snow module
PM module
```

Phase 12 — External Client Portal Integration Framework

Version:

```
v1.3.0-phase-12
```

Goal:

Create a generic integration framework for external client portals.

Core tables:

```
external_systems
external_accounts
external_credentials
```

```
external_work_order_links
external_status_mappings
external_priority_mappings
external_trade_mappings
external_sync_runs
external_sync_events
external_payload_logs
```

Expected folder pattern:

```
src/lib/integrations/
  core/
    types.ts
    push-client-update.ts
    sync-work-orders.ts
  servicechannel/
    client.ts
    mappers.ts
    sync-work-orders.ts
    push-status-update.ts
  other-provider/
    client.ts
    mappers.ts
```

Deliverables:

```
generic external system model
mapping tables
sync run logging
sync event logging
payload logging
first adapter skeleton
phase docs
```

Acceptance criteria:

```
Architecture supports multiple external portals.
ServiceChannel is not hardcoded into core jobs.
External statuses/priorities/trades can map to internal values.
Sync attempts are logged.
Failures can be reviewed.
Phase docs updated.
```

Do not build:

```
all external providers
email parser
full automation without review
```

Phase 13 — Email-to-Work-Order Ingestion

Version:

```
v1.4.0-phase-13
```

Goal:

Support clients who submit work orders by email or forwarded email.

Core tables:

```
email_ingestion_accounts
inbound_emails
email_parse_results
email_work_order_drafts
email_attachments
email_parser_rules
```

Deliverables:

```
inbound email storage model
email parse result model
work order draft intake model
operator review queue
approve draft into job
attachment linking
parse confidence tracking
phase docs
```

Acceptance criteria:

```
Inbound email can be stored.
Parser can extract draft work order data.
```

```
Draft does not automatically become active job unless approved.
Operator can review and approve draft intake.
Approved intake creates job.
Phase docs updated.
```

Do not build:

```
fully autonomous email job creation
advanced email routing for every client
```

Phase 14 — Preventative Maintenance Module

Version:

```
v1.5.0-phase-14
```

Goal:

Support recurring preventative maintenance programs.

Core tables:

```
pm_programs
pm_schedules
pm_schedule_locations
pm_assets
pm_visits
pm_visit_checklists
pm_visit_results
```

Deliverables:

```
create PM program
create PM schedule
assign client locations
generate PM jobs or draft visits
PM checklist templates
PM visit tracking
phase docs
```

Acceptance criteria:

```
Admin can create PM program.
Admin can assign locations.
System can create PM visit/job records.
PM jobs use source_type = preventative_maintenance.
PM jobs appear in normal job workflow where appropriate.
Phase docs updated.
```

Do not build:

```
snow operations
advanced asset lifecycle management unless needed
```

Phase 15 — Snow Operations Module

Version:

```
v1.6.0-phase-15
```

Goal:

Support snow/plowing workflows, which operate differently from normal break/fix jobs.

Core tables:

```
snow_programs
snow_sites
snow_service_triggers
snow_events
snow_event_sites
snow_dispatches
snow_service_logs
snow_weather_observations
```

Deliverables:

```
snow program model
snow site model
snow event model
```

```
batch dispatch concept
service log model
weather observation placeholder
snow dashboard section
phase docs
```

Acceptance criteria:

```
Snow program can be created.
Snow sites can be assigned.
Snow event can track affected locations.
Snow dispatch/service logs can be captured.
Snow operations are separate enough for batch workflows.
Phase docs updated.
```

Do not build:

```
full weather provider automation unless explicitly requested
```

Phase 16 — Chatbot and AI Operations Assistant

Version:

```
v2.0.0-phase-16
```

Goal:

Add a chatbot/AI assistant that understands the application, SOPs, workflows, and operational data.

Knowledge sources:

```
docs/
phase closeouts
chatbot knowledge docs
database schema
API route docs
business rules
user SOPs
```

```
admin SOPs
system workflows
```

AI use cases:

```
answer app usage questions
summarize job history
rewrite vendor notes into client-ready updates
recommend next action
identify stalled jobs
draft client updates
draft vendor follow-ups
summarize vendor performance
flag invoice anomalies
identify SLA risks
```

Deliverables:

```
chatbot knowledge search
job summary assistant
client update drafting
vendor follow-up drafting
AI action logging
source/citation pattern for internal docs if practical
phase docs
```

Acceptance criteria:

```
Chatbot can answer questions about app capabilities.
Chatbot can reference docs/SOPs.
AI can summarize job activity.
AI can draft reviewable updates.
AI actions are logged.
Phase docs updated.
```

Do not build:

```
uncontrolled autonomous operations
silent client updates without review
silent vendor dispatch without rules
```

---

9. Initial Table Plan by Domain

Future chats should treat this as planning guidance. Live schema should be inspected before implementing.

Admin / Tenancy

```
tenants
users
roles
user_roles
tenant_users
audit_logs
```

Clients / Locations

```
clients
client_contacts
client_locations
client_location_contacts
client_location_hours
client_location_access_notes
client_billing_rules
```

Vendors

```
vendors
vendor_contacts
vendor_locations
vendor_trade_coverage
vendor_service_areas
vendor_rates
vendor_documents
vendor_compliance
vendor_performance_scores
```

Jobs

```
jobs
job_contacts
job_status_history
job_priority_history
```

```
job_trade_history
job_notes
job_attachments
job_events
job_scope_steps
```

Dispatch

```
job_vendor_assignments
job_vendor_assignment_status_history
dispatch_messages
vendor_eta_confirmations
vendor_check_ins
vendor_check_outs
```

Communication

```
communication_logs
email_templates
outbound_messages
inbound_messages
client_update_logs
vendor_update_logs
portal_update_queue
```

AI Scope / AI Logging

```
scope_templates
scope_template_steps
ai_scope_generation_logs
ai_prompt_templates
ai_generated_updates
ai_action_logs
```

Billing / Proposals

```
proposals
proposal_line_items
proposal_approvals
change_orders
change_order_line_items
```

```
vendor_invoices
vendor_invoice_line_items
client_invoices
client_invoice_line_items
payment_records
job_billing_events
```

Reference Data

```
trades
priorities
job_statuses
countries
states
timezones
```

Integrations

```
external_systems
external_accounts
external_credentials
external_work_order_links
external_status_mappings
external_priority_mappings
external_trade_mappings
external_sync_runs
external_sync_events
external_payload_logs
```

Email Ingestion

```
email_ingestion_accounts
inbound_emails
email_parse_results
email_work_order_drafts
email_attachments
email_parser_rules
```

Preventative Maintenance

```
pm_programs
pm_schedules
pm_schedule_locations
pm_assets
pm_visits
pm_visit_checklists
pm_visit_results
```

Snow Operations

```
snow_programs
snow_sites
snow_service_triggers
snow_events
snow_event_sites
snow_dispatches
snow_service_logs
snow_weather_observations
```

---

10. Standard Phase Closeout Template

Each phase closeout should use this format.

```
# Phase X Closeout — <Phase Name>

## Phase Goal

<State the goal of the phase.>

## Completed Deliverables

- ...

## Files Created or Changed

- ...

## Database Changes
```

```
- ...

## API Routes / Server Actions Added

- ...

## User-Facing Workflows Added

- ...

## Admin/Internal Workflows Added

- ...

## Business Rules Added

- ...

## Chatbot Knowledge Added

- ...

## Verification Performed

Commands/results:

```bash

...
```

Known Limitations

- …

Carry-Forward Items

- …

Recommended Next Phase Focus

- …

```
A phase is not complete until this closeout exists.

---
```
```

````
# 11. Standard Next-Phase Handoff Template

When the user asks for a handoff prompt, produce this structure.

```text
We are starting Phase X of the PM Facilities Work Order Platform.

Project root:
~/Desktop/PM

Database:
MySQL through SSH tunnel on 127.0.0.1:3307 using database jonnyrosero_pm.

Session-safe MySQL command reminder:

ssh -p 21098 -L 3307:127.0.0.1:3306 jonnyrosero@host62.registrar-
servers.com

Then in another terminal:

cd ~/Desktop/PM 2>/dev/null || cd ~/Desktop/pm
read -s MYSQL_PWD
export MYSQL_PWD
mysql --protocol=tcp -h 127.0.0.1 -P 3307 -u jonnyrosero_jonny
jonnyrosero_pm -e "SELECT DATABASE() AS db_name, NOW() AS server_time;"

Important project rules:
- Work step by step in small batches.
- Stay inside Phase X scope unless there is a true blocker or explicit
instruction.
- Do not make the app ServiceChannel-specific.
- The platform is source-agnostic.
- Aggregator portal is first priority.
- Vendor portal and client portal are future first-class portals.
- Vendor updates should be captured and reviewed/mapped before client
sharing where appropriate.
- AI-assisted scope generation comes before full AI automation.
- Email ingestion is a future requirement but should not be overbuilt
early.
- Every meaningful operational change should preserve history/event
records.
- Every phase must update closeout docs and chatbot knowledge docs.

Source-of-truth order:
1. Current user instruction.
2. GPT project roadmap.
3. Live repo.
4. Live DB schema.
````
````

```
5. Current phase docs.
6. Older docs for historical context only.

Phase X goal:
<insert phase goal>

Required deliverables:
<insert phase deliverables>

Do not build yet:
<insert out-of-scope items>

Before editing:
- Inspect repo structure.
- Inspect relevant files.
- Inspect live database schema if DB work is involved.
- Summarize findings.
- Propose the first small implementation batch.
```

---

12. Phase 1 Starting Prompt

Use this prompt after Phase 0 is committed and docs are uploaded.

```
We are starting Phase 1 of the PM Facilities Work Order Platform.

Project root:
~/Desktop/PM

Database:
MySQL through SSH tunnel on 127.0.0.1:3307 using database jonnyrosero_pm.

Important rules:
- Work step by step in small batches.
- Do not make the app ServiceChannel-specific.
- The platform is source-agnostic: internal client portal, external portals,
email ingestion, manual entry, API, PM schedules, and snow events are all
possible job sources.
- Aggregator portal is first priority.
- Vendor portal and client portal are future first-class portals.
- Vendor updates should be captured and reviewed/mapped before being shared with
clients where appropriate.
- AI-assisted scope generation is an early planned feature, but full chatbot/
automation comes later.
- Email ingestion is a future requirement, but do not overbuild it in Phase 1.
```

```
- Every meaningful workflow should preserve history/auditability.
- Every phase must update closeout docs and chatbot knowledge docs.

Source-of-truth order:
1. Current instruction.
2. GPT project roadmap.
3. Live repo.
4. Live DB schema.
5. Current phase docs.
6. Older docs for historical context only.

Phase 1 goal:
Build the multi-tenant foundation, users, roles, and auth.

Required deliverables:
- tenants table
- users table
- roles table
- tenant_users or equivalent membership table
- user_roles or equivalent role assignment table
- audit_logs table if practical
- protected app shell/dashboard
- login/logout flow
- tenant-aware server-side data access pattern
- Phase 1 docs under docs/phase-1-auth-tenancy/

Do not build yet:
- clients
- client locations
- vendors
- jobs
- dispatch
- portals
- integrations
- email parser
- AI scope generation

Before editing, inspect the current repo structure, package setup, and database
state. Then summarize findings and propose the first small implementation batch.
```

---

13. Project Alignment Reminder

Future GPT chats should always remember:

The goal is not to build isolated screens.

The goal is to build a structured operating platform where every job, dispatch, note, status, vendor update, client update, invoice, proposal, and integration event becomes part of one auditable operational system.

The app should help the aggregator move from reactive manual coordination to structured, trackable, AI-assisted facilities operations.